

Using the shell

For names without /, the shell searches the directories in PATH (separated by ;; default . /user). A path containing / is used as written. The shell loads the program, starts it, and prints its PID.

```
PicoOS> echo.bin hello
process with pid 3 created
hello
```

Max line: **79 characters**. Spaces and tabs split arguments. \$NAME, \$? (last foreground result), and \$! (latest background PID) are replaced before the program starts. Double quotes are removed but do not keep spaces inside one argument. Names and paths are case-sensitive.

All built-ins

```
cd DIR change the current directory
export N=V set a variable for child programs
load PATH create NEW; print PID
run PID ARGS start loaded process
unload PID remove one not yet run
list print PID binary-path for all processes
fg / bg continue the latest job
eval CMD run another shell command
exit end shell; init restarts it
run-shell-tests FILE run a shell-test file
```

Editing, history, environment

```
Enter / Ctrl+M submit
Backspace / Delete erase char
Ctrl+U / Ctrl+W erase line / word
↑ / ↓ last 8 distinct commands
Left/Right do not move the cursor; other special key sequences do nothing.
Programs receive a copy of the shell environment. Init reads ./config/
environment.txt. N=V alone is treated as a command; there is no shell unset.
```

What the shell cannot do

There are no pipes, input redirection (<), error redirection (>), wildcards, aliases, \$(...), several commands separated by ;, single quotes, or quotes that preserve spaces. PICOOS_LOADING_BAR=true shows loading bars for supported file and program transfers.

User applications

```
echo.bin TEXT... print args; literal \n → newline
cat.bin FILE... print files; does not read keyboard input
count.bin [DELAY] count forever; default 25000
ls.bin [DIR] list dir; default .
pwd.bin print cwd
mkdir.bin DIR... create directories
rm.bin FILE... remove files
rmdir.bin DIR... remove empty dirs
kill.bin [SIG] PID default SIGTERM
poweroff.bin halt PicoOS/emulator
Utilities support -h / --help where applicable. cat, mkdir, rm, and rmdir keep trying
the remaining paths after an error.
```

Redirection

```
CMD > FILE create or empty the file; write output there
CMD >> FILE create the file or add output at the end

echo.bin first > log.txt
echo.bin next >> log.txt
count.bin 0 >> counts.txt &
```

Put a space before > / >>, and put the file name last. This works with programs, run, and &. Error messages stay on the terminal.

ls.bin & host files

Output is d NAME for a directory or - NAME for a regular file. It includes ., .., and hidden names; no size, owner, permissions, sort, or flags. Order is the host directory order. PicoOS has no disk of its own. It asks the emulator through UART to read and write files on the host computer. File descriptors 0/1/2 are stdin/stdout/stderr; regular files use 3–7. Child programs inherit the three standard descriptors and the current directory. PicoOS understands ., .., and repeated slashes. File writes add at the end; > empties the file first. l seek changes where the next read starts. PATH= . /user still points to the original user directory after cd.

Files and directories in programs

```
read(fd,buf,n) · write(fd,buf,n)
close(fd) · dup2(old,new)
lseek(fd,off,SEEK_SET|CUR|END)
open(path,flags,...) · creat(path,mode)
chdir(path) · getcwd(buf,size)
unlink(path) · rmdir(path) · mkdir(path)
```

Flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_TRUNC, O_APPEND; O_ACCMODE selects the read/write part. PicoOS ignores Unix permission modes. PATH_MAX=512. A negative return means an error; read and write may handle fewer than n cells. Descriptors 0–2 cannot be closed. Include functions with library/<name>/<name>.header; PicoOS implements only part of POSIX.

Directories · dirent

```
DIR *opendir(path)
struct dirent *readdir(DIR*)
closedir(DIR*)
DT_DIR · DT_REG · DIRENT_NAME_MAX=128
```

readdir returns NULL at the end and reuses the same entry each time. Copy a name if it is needed after the next call. opendir can hold up to 512 cells and includes . and ...

Streams · stdio

```
stdin · stdout · stderr
fopen(path,mode) · fclose(FILE*)
fputc(c,f) · fputs(text,f)
fprintf(f,fmt,...) · printf(fmt,...)
scanf(fmt,...)
```

Five extra streams can be open. Output is written immediately. Modes are r, w, a, and the same modes with +. printf/fprintf support %d %c %s %%; scanf supports the same. %s has no length limit, so its destination buffer must be large enough.

Strings · string

```
memcpy(dst,src,n) · memset(buf,val,n)
strcpy(dst,src) · strcat(dst,src)
strcmp(a,b) · strncmp(a,b,n) · strlen(s)
```

RETl/PicoC stores characters, integers, and pointers in 32-bit cells. Sizes and counts therefore refer to cells, not bytes on the host computer.

Jobs & process actions

A trailing & starts a program without waiting. fg continues the latest job in the foreground and waits for it; bg continues it in the background. The shell remembers only one background or stopped job—there is no jobs list or %job syntax.

```
count.bin 0 &
echo.bin pid=$!
fg          # then Ctrl+Z
bg
```

load creates a process, run starts it, and unload removes it only if it has not run. l shows every PID and program path. Result 0 means success; utilities normally use 1 for bad input or another error.

Terminal signals

```
Ctrl+C SIGTERM 15; terminate by default
Ctrl+Z SIGTSTP 20; stop by default
These keys send a signal to the foreground program. In the emulator debugger, use capital V so the keys reach PicoOS.
0 check whether the process exists; send nothing
SIGKILL 9 always terminate; cannot be handled or ignored
SIGTERM 15 terminate; may be handled or ignored
SIGCHLD 17 child notice; ignored by default
SIGCONT 18 continue
SIGTSTP 20 stop; may be handled or ignored
kill.bin accepts these names or numbers. Signal status is 128 + sig.
```

Processes in programs

```
load(path) · run(pid,args,env)
unload(pid) · list() · getpid()
waitpid(pid) · WIFSTOPPED(status) · exit(status)
```

With run(... , NULL), the child gets a copy of the current environment. waitpid(pid) waits for that child and returns its result. There is no options argument and no waitpid(-1). Returning from main calls exit.

Memory and program start

```
malloc(size) · realloc(ptr,size) · free(ptr)
atoi(text) · exit(status)
```

Before main, _start sets up the heap and environment. It then calls main(argc,argv) and passes the return value to exit. The heap size is fixed. PicoOS has no virtual memory or memory protection.

Environment · stdlib

```
getenv(name) · setenv(name,value,overwrite)
unsetenv(name) · putenv("N=V") · clearenv()
```

Variables are stored in the process heap, and putenv copies its text. Child programs receive a copy. Do not keep a pointer returned by getenv after changing the environment. setenv(... , false) keeps an existing value.

Signals & parent death

```
signal(sig,handler) · kill(pid,sig)
SIG_DFL · SIG_IGN · SIG_ERR
prctl(PR_SET_PDEATHSIG, sig)
```

A program cannot handle or ignore SIGKILL; it can do so for the other listed signals. Only one signal handler can run at a time, so keep it short. Parent-death signal 0 disables the setting. This is the only supported prctl option.

Scheduling & mutexes

```
yield()
mutex_init(m) · mutex_lock(m) · mutex_unlock(m)
```

The timer switches between ready programs in turn. yield lets another program run early. A mutex waits instead of repeatedly checking the lock; it uses RETl's atomic TSL instruction and wakes waiters in order. PicoOS does not switch processes while kernel code is running.

Shared memory · sys/mman

```
shm_open(name,size) · mmap(id)
shm_unlink(name)
```

shm_open creates or opens a named block; its size is in 32-bit cells. mmap returns the shared address. Use a mutex when several programs write it. This mmap is not connected to files and accepts only the shared ID.

Project build

```
make help show targets
make user build user binaries
make firmware build bootloader and kernel files
make release-tree build everything under binary/
make verify-release-tree check that all release files exist
make bootload build + boot in debugger
make bootload-debug rebuild -g + boot
Layers: make eprom, kernel, isrs, system.
```

Compile and size a program

Build uses picoc_compiler --show-input-files, -C startup, -O1, -s, -g, -o; debug also uses -i -w -v. -k sram|eprom writes the memory constants header. Prefer Make; use -h for the full option list.

```
picoc_compiler --heap-size 2000 \
--stack-size 1000 -o app.reti app.picoc
reti_emulator -a app.reti
```

-a reads app.sections, adds the five layout numbers PicoOS needs, writes app.bin, and exits. You may edit .sections after compiling and before assembling. For N stack cells, use stack_start = heap_start + heap_size + N. The stack grows downward and must not overlap the heap.

Use the emulator debugger

Make uses emulator -e EPROM, -n 4 IVT entries, -r SRAM cells, -S sections, -D debuginfo, -d debugger, -c comments, -0 first RTl. Press c to run. Capital V opens the raw UART terminal; Ctrl+] returns. This mode sends control keys and arrows to PicoOS. E pauses, n runs one instruction, r restarts, lowercase v opens the normal UART view (Escape returns), and q closes a menu or quits.